Full chat with Claude

Share link: https://claude.ai/share/ffe2ae1b-6e30-4c3f-b39c-2e1af5617c43

PROMPT 1

```
Hi! I'm a computer science student and I'm working on an OOP assignment
in C++. I need to design a simulator for a made-up computer called the
Vole machine (it comes from Brookshear's "Computer Science: An Overview"
book). For now I only need the DESIGN, not the code. I want a UML class
diagram written in PlantUML.

Let me explain the machine first.

The Vole machine has:
- A memory of 256 cells. Each cell holds 1 byte, and the addresses go from
  00 to FF in hex. The cell at address FF is special: it is connected to
  the screen, so writing to FF means printing to the screen.
- 16 general-purpose registers, numbered 0 to F, each holding 1 byte.
- A Program Counter (PC), which holds the address of the next instruction,
  and an Instruction Register (IR), which holds the current instruction.
- Every instruction is 2 bytes (4 hex digits), so it takes two memory cells.
- The CPU repeats a cycle: fetch the instruction at PC into IR (and move PC
  forward by 2), decode it, then execute it.

The instructions I need to support are (R, S, T are register numbers, and
XY is a hex value or address):
1 RXY: load register R from memory cell XY
2 RXY: load register R with the value XY
3 RXY: store register R into memory cell XY
3 RFF: store register R to cell FF, which means print to the screen
4 0RS: copy register R into register S
5 RST: add registers S and T as two's complement integers, put the result in R
6 RST: add registers S and T as floating-point numbers, put the result in R
7 RST: R = S OR T
8 RST: R = S AND T
9 RST: R = S XOR T
A R0X: rotate register R to the right X times (circular rotation)
B RXY: if register R equals register 0, jump to address XY (set PC = XY),
       otherwise just continue
C 000: halt

Here is what the simulator should do:
1. Show a menu of choices.
2. Load a program from a text file into memory, starting at an address the
   user chooses. That address must be an even number (00, 02, 04, ..., F0,
   F2, ...) and less than FF.
3. Let the user overwrite any register or any memory cell with a new value.
4. Set the PC to the start address of the program.
5. Show the memory as a 16x16 table, plus the PC, the IR and all 16 registers.
6. Run the program by fetching each instruction into IR, checking that it
   is a valid instruction, decoding it, executing it and updating the PC.
7. Let the user choose between running the whole program at once or step
   by step.
```

8. If it runs all at once, show the memory, PC, IR, registers and the screen
   output when it finishes.
9. If it runs step by step, show all of that after every step, and wait
   for the user to press Enter before the next step.
10. When the program finishes, ask the user if they want to exit or load
    another program.
11. Invalid instructions must be rejected, and a program that contains
    invalid instructions must be rejected too.

The program file has one instruction per line, written as 4 hex digits
(for example 2005). Lines that start with ; are comments and should be
ignored, empty lines are ignored too, and anything after the 4 hex digits
on a line is ignored.

I already tried to design this myself. This is what I came up with:
Classes:
1. Memory
   - Responsibility: stores the 256 memory cells.
   - Attributes: cells (array of 256 bytes)
   - Methods: read(address), write(address, value), clear()

Register
Responsibility: stores the 16 general-purpose registers.
Attributes: regs (array of 16 bytes)
Methods: read(index), write(index, value), clear()

CPU
Responsibility: runs the fetch-decode-execute cycle.
Attributes: programCounter, instructionRegister, registers (Register), memory
(Memory)
Methods: fetch(), decode(), execute(), step(), run()

ALU
Responsibility: does the arithmetic and logic operations.
Methods: addInt(a, b), addFloat(a, b), orOp(a, b), andOp(a, b), xorOp(a, b),
rotate(value, times)

Machine
Responsibility: owns the CPU and the memory, and loads programs.
Attributes: cpu (CPU), memory (Memory)
Methods: loadProgram(filename, startAddress), run(), runStep(),
overwriteRegister(index, value), overwriteMemory(address, value)

UI
Responsibility: the menu and all printing.
Attributes: machine (Machine)
Methods: showMenu(), displayMemory(), displayRegisters(), displayScreen(), start()

Relations:
- Machine has one CPU and one Memory (composition).
- CPU has one Register set and one ALU (composition) and uses Memory (association).
- UI uses Machine (association).

Could you please:
1. Look at my design and tell me honestly what is good and what could be

```
   better.
2. Give me your own best design as PlantUML code, with the classes,
   attributes, methods (including parameter and return types), visibility
   and the relationships between classes (inheritance, composition,
   association, dependency), with multiplicities where they make sense.
3. Explain in simple words what each class is responsible for and why you
   chose to structure it this way.

Please try to follow good OOP principles like encapsulation and single and send it as
a text not as a file
responsibility. Thanks!
```

RESPONSE 1

1. Review of your design

What's good

- You split the machine along hardware lines (memory, registers, ALU, CPU), so each class is small and cohesive.
- The UI is separate from the logic, so the model never calls cout.
- Composition is used where the part cannot live without the whole (Machine → CPU/Memory, CPU → registers). Having the CPU only use a Memory owned by Machine is also right.

What could be better

- There is no Instruction concept. Your decode() has nothing to return, and execute() would become a big switch over 12 opcodes. That makes the CPU a "god class" and breaks the Open/Closed principle. Validation (requirements 6 and 11) also has no home.
- There is no Screen class. displayScreen() is in the UI, but the screen output has to be stored somewhere. Writing to cell FF is memory-mapped I/O, which is hardware behaviour and should live in the model, not in the UI.
- Machine does too much. Reading and validating a program file is its own responsibility. It also needs to be all-or-nothing: if any line is invalid, nothing should be written to memory.
- UI mixes input handling (menu) with output formatting (tables). I would split these into two classes.
- Register is a misleading name. It is really a register file (16 registers). PC and IR as plain public ints would break encapsulation, so keep them private with accessors. IR is 2 bytes wide, not 1.
- ALU has no state, so "CPU owns an ALU" (composition) describes a lifetime relationship that doesn't exist. Make it a utility class with static methods and use a dependency instead.
- Error handling is not designed. Invalid instructions and bad files should be reported with dedicated exception types.
- Types are vague. Use uint8_t and uint16_t so a "byte" is really a byte.

2. PlantUML design

```
@startuml VoleMachine
skinparam classAttributeIconSize 0
skinparam shadowing false
hide empty members
```

```
' ====================== HARDWARE ======================
package "Hardware" {

  class Screen {
    - buffer : std::string
    + print(value : uint8_t) : void
    + getContent() : const std::string&
    + clear() : void
  }

  class Memory {
    + {static} SIZE : size_t = 256
    + {static} SCREEN_ADDRESS : uint8_t = 0xFF
    - cells : std::array<uint8_t, 256>
    - screen : Screen&
    + Memory(screen : Screen&)
    + read(address : uint8_t) : uint8_t
    + write(address : uint8_t, value : uint8_t) : void
    + clear() : void
  }

  class RegisterFile {
    + {static} COUNT : size_t = 16
    - regs : std::array<uint8_t, 16>
    + read(index : uint8_t) : uint8_t
    + write(index : uint8_t, value : uint8_t) : void
    + clear() : void
  }

  class ALU <<utility>> {
    + {static} addTwosComplement(a : uint8_t, b : uint8_t) : uint8_t
    + {static} addFloat(a : uint8_t, b : uint8_t) : uint8_t
    + {static} bitOr(a : uint8_t, b : uint8_t) : uint8_t
    + {static} bitAnd(a : uint8_t, b : uint8_t) : uint8_t
    + {static} bitXor(a : uint8_t, b : uint8_t) : uint8_t
    + {static} rotateRight(value : uint8_t, times : uint8_t) : uint8_t
    - {static} floatToReal(bits : uint8_t) : double
    - {static} realToFloat(x : double) : uint8_t
  }

  interface ExecutionContext {
    + {abstract} readRegister(index : uint8_t) : uint8_t
    + {abstract} writeRegister(index : uint8_t, value : uint8_t) : void
    + {abstract} readMemory(address : uint8_t) : uint8_t
    + {abstract} writeMemory(address : uint8_t, value : uint8_t) : void
    + {abstract} setProgramCounter(address : uint8_t) : void
    + {abstract} halt() : void
  }

  class CPU {
    - pc : uint8_t
    - ir : uint16_t
    - halted : bool
    - registers : RegisterFile
    - memory : Memory&
```

```
    - current : std::unique_ptr<Instruction>
    + CPU(memory : Memory&)
    + restart(startAddress : uint8_t) : void
    + step() : void
    + run() : void
    + isHalted() : bool
    + getPC() : uint8_t
    + getIR() : uint16_t
    + getRegisters() : const RegisterFile&
    + readRegister(index : uint8_t) : uint8_t
    + writeRegister(index : uint8_t, value : uint8_t) : void
    + readMemory(address : uint8_t) : uint8_t
    + writeMemory(address : uint8_t, value : uint8_t) : void
    + setProgramCounter(address : uint8_t) : void
    + halt() : void
    - fetch() : void
    - decode() : void
    - execute() : void
  }
}

' ==================== INSTRUCTIONS ====================
package "Instructions" {

  abstract class Instruction {
    # raw : uint16_t
    # Instruction(raw : uint16_t)
    + getRaw() : uint16_t
    + toHexString() : std::string
    + {abstract} execute(ctx : ExecutionContext&) : void
  }

  abstract class BinaryRegisterInstruction {
    # r : uint8_t
    # s : uint8_t
    # t : uint8_t
    + execute(ctx : ExecutionContext&) : void
    # {abstract} compute(a : uint8_t, b : uint8_t) : uint8_t
  }

  class LoadFromMemory {
    - r : uint8_t
    - address : uint8_t
    + execute(ctx : ExecutionContext&) : void
  }
  class LoadImmediate {
    - r : uint8_t
    - value : uint8_t
    + execute(ctx : ExecutionContext&) : void
  }
  class StoreToMemory {
    - r : uint8_t
    - address : uint8_t
    + execute(ctx : ExecutionContext&) : void
  }
```

```
  class CopyRegister {
    - r : uint8_t
    - s : uint8_t
    + execute(ctx : ExecutionContext&) : void
  }
  class AddTwosComplement {
    # compute(a : uint8_t, b : uint8_t) : uint8_t
  }
  class AddFloat {
    # compute(a : uint8_t, b : uint8_t) : uint8_t
  }
  class OrRegisters {
    # compute(a : uint8_t, b : uint8_t) : uint8_t
  }
  class AndRegisters {
    # compute(a : uint8_t, b : uint8_t) : uint8_t
  }
  class XorRegisters {
    # compute(a : uint8_t, b : uint8_t) : uint8_t
  }
  class RotateRight {
    - r : uint8_t
    - times : uint8_t
    + execute(ctx : ExecutionContext&) : void
  }
  class JumpIfEqual {
    - r : uint8_t
    - address : uint8_t
    + execute(ctx : ExecutionContext&) : void
  }
  class Halt {
    + execute(ctx : ExecutionContext&) : void
  }

  class InstructionDecoder <<factory>> {
    + {static} decode(high : uint8_t, low : uint8_t) : std::unique_ptr<Instruction>
    + {static} isValid(high : uint8_t, low : uint8_t) : bool
  }

  class "std::runtime_error" as RuntimeError <<std>>
  class InvalidInstructionException {
    + InvalidInstructionException(raw : uint16_t)
  }
}

' ==================== LOADER ====================
package "Loader" {
  class ProgramLoader {
    + load(filename : const std::string&, startAddress : uint8_t, memory : Memory&) :
uint8_t
    - parseFile(filename : const std::string&) : std::vector<uint16_t>
    - parseLine(line : const std::string&) : std::optional<uint16_t>
    - checkStartAddress(startAddress : uint8_t, wordCount : size_t) : void
  }
  class ProgramLoadException {
```

```
    + ProgramLoadException(message : const std::string&)
  }
}

' ====================== FACADE ======================
class Machine <<facade>> {
  - screen : Screen
  - memory : Memory
  - cpu : CPU
  - loader : ProgramLoader
  - programStart : uint8_t
  + Machine()
  + loadProgram(filename : const std::string&, startAddress : uint8_t) : void
  + overwriteRegister(index : uint8_t, value : uint8_t) : void
  + overwriteMemory(address : uint8_t, value : uint8_t) : void
  + setPC(address : uint8_t) : void
  + setPCToProgramStart() : void
  + step() : bool
  + runAll() : void
  + isFinished() : bool
  + getMemory() : const Memory&
  + getCPU() : const CPU&
  + getScreen() : const Screen&
}

' ====================== UI ======================
package "UI" {
  class ConsoleUI {
    - machine : Machine&
    - renderer : ConsoleRenderer
    + ConsoleUI(machine : Machine&)
    + start() : void
    - showMenu() : void
    - handleLoadProgram() : void
    - handleOverwriteRegister() : void
    - handleOverwriteMemory() : void
    - handleSetPC() : void
    - handleRun() : void
    - runAllAtOnce() : void
    - runStepByStep() : void
    - askExitOrLoadAnother() : bool
    - readHex(prompt : const std::string&) : uint8_t
    - readLine(prompt : const std::string&) : std::string
  }

  class ConsoleRenderer {
    + showState(machine : const Machine&) : void
    + showMemory(memory : const Memory&) : void
    + showRegisters(regs : const RegisterFile&) : void
    + showPCAndIR(pc : uint8_t, ir : uint16_t) : void
    + showScreen(screen : const Screen&) : void
    + showMessage(text : const std::string&) : void
    + showError(text : const std::string&) : void
  }
}
```

```
' ==================== RELATIONSHIPS ====================
' Ownership
Machine "1" *-- "1" Screen
Machine "1" *-- "1" Memory
Machine "1" *-- "1" CPU
Machine "1" *-- "1" ProgramLoader
CPU "1" *-- "1" RegisterFile
CPU "1" *-- "0..1" Instruction : current

' Associations
CPU "1" --> "1" Memory : reads / writes
Memory "1" --> "1" Screen : prints when\naddress = FF

' Interface realisation
CPU ..|> ExecutionContext

' Instruction hierarchy
Instruction <|-- LoadFromMemory
Instruction <|-- LoadImmediate
Instruction <|-- StoreToMemory
Instruction <|-- CopyRegister
Instruction <|-- BinaryRegisterInstruction
Instruction <|-- RotateRight
Instruction <|-- JumpIfEqual
Instruction <|-- Halt
BinaryRegisterInstruction <|-- AddTwosComplement
BinaryRegisterInstruction <|-- AddFloat
BinaryRegisterInstruction <|-- OrRegisters
BinaryRegisterInstruction <|-- AndRegisters
BinaryRegisterInstruction <|-- XorRegisters

' Dependencies
Instruction ..> ExecutionContext : uses
AddTwosComplement ..> ALU
AddFloat ..> ALU
OrRegisters ..> ALU
AndRegisters ..> ALU
XorRegisters ..> ALU
RotateRight ..> ALU
CPU ..> InstructionDecoder : decodes with
InstructionDecoder ..> Instruction : creates
InstructionDecoder ..> InvalidInstructionException : throws
ProgramLoader ..> InstructionDecoder : validates with
ProgramLoader ..> Memory : writes program to
ProgramLoader ..> ProgramLoadException : throws

' Exceptions
RuntimeError <|-- InvalidInstructionException
RuntimeError <|-- ProgramLoadException

' UI
ConsoleUI "1" --> "1" Machine
ConsoleUI "1" *-- "1" ConsoleRenderer
```

```
ConsoleRenderer ..> Machine : reads state of
@enduml
```
(Constructors of the concrete instruction classes are left out to keep the diagram readable. Each one takes the decoded operands.)

3. What each class does and why

Hardware

- **Screen:** stores everything the program has printed. Keeping it in the model means the UI only has to display it.
- **Memory:** 256 cells with read/write. It knows that address FF is the screen, so a write to FF goes to Screen::print. Because of this, the instruction 3 RFF needs no special case, since it is just a normal store.
- **RegisterFile:** the 16 registers. It checks that the index is 0 to F and throws std::out_of_range otherwise.
- **ALU:** pure calculations with no state, so the methods are static. addFloat needs two private helpers because Brookshear's float is 1 sign bit, a 3-bit exponent (excess-4) and a 4-bit mantissa.
- **ExecutionContext:** a small interface listing exactly what an instruction may do to the machine (read and write registers and memory, jump, halt). Instructions depend on this interface and not on CPU, which avoids a circular dependency and keeps the CPU's internals hidden.
- **CPU:** owns PC, IR and the registers, and runs one cycle in step(): fetch() reads 2 bytes at PC into IR and does PC += 2, decode() asks the factory for an Instruction (which validates it), and execute() calls current->execute(*this). run() repeats step() until halted. It doesn't know what any opcode means.

Instructions

- **Instruction (abstract):** one polymorphic method, execute(ctx). Each of the 12 opcodes is its own small subclass, so the CPU has no switch. Adding an opcode means adding a class (Open/Closed principle).
- **BinaryRegisterInstruction:** opcodes 5 to 9 share the shape R = S op T. This class implements execute once (Template Method) and the subclasses only supply compute(a, b), which avoids duplicated code.
- **InstructionDecoder (factory):** the only place that knows the opcode table and the validity rules. Examples: opcode 0 and D, E, F are invalid. C must be exactly C000. 4 must be 40RS. A must be AR0X. It throws InvalidInstructionException otherwise.
- **Exceptions:** they give clear, typed error reporting instead of return codes.

Loader

- **ProgramLoader:** reads the file, skips ; comments and empty lines, takes the first 4 hex digits of each line, and checks every instruction with the decoder. It also checks that the start address is even and below FF and that the program fits before FF. Only if everything is valid does it write to memory, so a bad program is rejected as a whole and memory is never half-loaded.

Facade and UI

- **Machine:** the single entry point the UI talks to. It owns the Screen, Memory, CPU and loader, so it controls their lifetimes. step() returns true while the program is still running. It exposes only const views for display.
- **ConsoleUI:** the menu and user input (parsing hex, handling exceptions, the Enter-to-continue loop for step mode, “exit or load another?”).
- **ConsoleRenderer:** all formatting (16×16 memory table, registers, PC, IR, screen). Changing the look, or replacing it with a GUI, does not touch the input logic.

Why this structure follows OOP principles

- Single responsibility: parsing, decoding, executing, storing and displaying are each in their own class.
- Encapsulation: all data members are private or protected, and the outside only sees a small public interface.
- Polymorphism and inheritance: the instruction hierarchy replaces conditionals.
- Dependency inversion: instructions depend on the ExecutionContext interface, not on the concrete CPU.
- Low coupling: the UI only knows Machine, and the model never prints.

Details to decide before coding

- C++ initialization order: in Machine, declare members in the order screen, memory, cpu, because Memory holds a reference to Screen and CPU to Memory.
- Program end near FF: I made the loader reject programs that would reach cell FF. Otherwise loading would print to the screen.
- Odd jump addresses and runaway programs: B RXY with an odd XY makes the following fetches misaligned. You can either allow it, as the original machine does, or reject it as invalid. Also consider a maximum-step guard in runAll() to stop infinite loops.
- Screen output: print the byte as an ASCII character. You may also want to show it in hex.

I can also draw a sequence diagram of one step() or write the file-parsing rules in more detail if that would help.

PROMPT 2

```
Refactor the code toward the following architecture, adapting it to the existing
project where appropriate:
Contains 256 bytes, uses uint8_t addresses and values, and implements memory-mapped
screen output at address 0xFF.
```

RESPONSE 2

I’ll check whether there’s existing code in the workspace to refactor.

There’s no code in the workspace yet (only the design from earlier), so there’s nothing to refactor. Here is the Memory part of the architecture written from scratch. It matches the PlantUML design and uses uint8_t for addresses and values, so every possible address is valid by construction and needs no range check.

[The C++ code of this answer (Screen.h, Memory.h and Memory.cpp) is not reproduced in this document. See the share link.]

Design choices

- **Memory-mapped screen:** Memory::write is the only place that knows about 0xFF. Instruction 3 RFF is therefore an ordinary StoreToMemory, with no special case in the CPU or in the instruction classes.
- **Value kept at FF:** The value is also stored in the cell, so reading FF returns the last character written. If you want FF to be write-only, skip the cells_ assignment for that address.
- **Reference to Screen:** Memory holds a Screen&, so Screen must outlive it. In Machine, declare screen before memory so the construction order is correct.
- **clear() and the screen:** clear() resets only the cells. Clear the screen separately, for example when a new program is loaded.

If you have existing code you'd like refactored toward this design, paste it here and I'll adapt it. I can also write the next classes (RegisterFile, ALU, CPU) in the same style.

PROMPT 3

```
send a full code as a text like first one
```

RESPONSE 3

Here is the full C++17 code for the design, split by file. Nothing is saved as a file; copy each block into a file with the name shown above it.

[The C++ source code of this answer (24 files: Hex.h, Exceptions.h, Screen.h, Memory.h, Memory.cpp, RegisterFile.h, ALU.h, ALU.cpp, ExecutionContext.h, Instruction.h, Instructions.h, InstructionDecoder.h, InstructionDecoder.cpp, CPU.h, CPU.cpp, ProgramLoader.h, ProgramLoader.cpp, Machine.h, Machine.cpp, ConsoleRenderer.h, ConsoleRenderer.cpp, ConsoleUI.h, ConsoleUI.cpp and main.cpp), the build commands and the test program are not reproduced in this document. See the share link.]

Small differences from the UML

- **CPU::run(maxSteps):** It has a step limit (default 100000), so an infinite loop ends with an error instead of freezing.
- **reset() / resetAll():** I added them, with menu option 7 to use them.
- **ConsoleUI::handleRun():** It returns bool, so the "exit or load another" answer can end the program.
- **Error handling in CPU::step():** After any error, the CPU is marked as halted, so the UI knows the run is over.
- **Hex.h and Instructions.h:** Hex.h is a helper that formats numbers as hex. Instructions.h groups all 12 small instruction classes in one header.
- **Invalid-instruction errors from the loader:** They include the line number, for example Line 3: invalid instruction D123.

I can also explain any class in more detail or help you write unit tests for the ALU and the decoder.

PROMPT 4

```
ok send that one can I upload it on plantuml and send it as a text
```

RESPONSE 4

Here is the PlantUML code, updated to match the final C++ code (reset(), resetAll(), the maxSteps limit, and handleRun() returning bool). Copy it all and paste it into the editor at https://www.plantuml.com/plantuml.

```
@startuml VoleMachine
skinparam classAttributeIconSize 0
skinparam shadowing false
hide empty members

' ==================== HARDWARE ====================
package "Hardware" {

  class Screen {
    - buffer : std::string
    + print(value : uint8_t) : void
    + getContent() : const std::string&
    + clear() : void
  }

  class Memory {
    + {static} SIZE : size_t = 256
    + {static} SCREEN_ADDRESS : uint8_t = 0xFF
    - cells : std::array<uint8_t, 256>
    - screen : Screen&
    + Memory(screen : Screen&)
    + read(address : uint8_t) : uint8_t
    + write(address : uint8_t, value : uint8_t) : void
    + clear() : void
  }

  class RegisterFile {
    + {static} COUNT : size_t = 16
    - regs : std::array<uint8_t, 16>
    + read(index : uint8_t) : uint8_t
    + write(index : uint8_t, value : uint8_t) : void
    + clear() : void
    - {static} checkIndex(index : uint8_t) : void
  }

  class ALU <<utility>> {
    + {static} addTwosComplement(a : uint8_t, b : uint8_t) : uint8_t
    + {static} addFloat(a : uint8_t, b : uint8_t) : uint8_t
    + {static} bitOr(a : uint8_t, b : uint8_t) : uint8_t
    + {static} bitAnd(a : uint8_t, b : uint8_t) : uint8_t
    + {static} bitXor(a : uint8_t, b : uint8_t) : uint8_t
    + {static} rotateRight(value : uint8_t, times : uint8_t) : uint8_t
    - {static} floatToReal(bits : uint8_t) : double
    - {static} realToFloat(x : double) : uint8_t
  }
```

```
  interface ExecutionContext {
    + {abstract} readRegister(index : uint8_t) : uint8_t
    + {abstract} writeRegister(index : uint8_t, value : uint8_t) : void
    + {abstract} readMemory(address : uint8_t) : uint8_t
    + {abstract} writeMemory(address : uint8_t, value : uint8_t) : void
    + {abstract} setProgramCounter(address : uint8_t) : void
    + {abstract} halt() : void
  }

  class CPU {
    - pc : uint8_t
    - ir : uint16_t
    - halted : bool
    - registers : RegisterFile
    - memory : Memory&
    - current : std::unique_ptr<Instruction>
    + CPU(memory : Memory&)
    + restart(startAddress : uint8_t) : void
    + reset() : void
    + step() : void
    + run(maxSteps : size_t = 100000) : void
    + isHalted() : bool
    + getPC() : uint8_t
    + getIR() : uint16_t
    + getRegisters() : const RegisterFile&
    + readRegister(index : uint8_t) : uint8_t
    + writeRegister(index : uint8_t, value : uint8_t) : void
    + readMemory(address : uint8_t) : uint8_t
    + writeMemory(address : uint8_t, value : uint8_t) : void
    + setProgramCounter(address : uint8_t) : void
    + halt() : void
    - fetch() : void
    - decode() : void
    - execute() : void
  }
}

' ==================== INSTRUCTIONS ====================
package "Instructions" {

  abstract class Instruction {
    # raw : uint16_t
    # Instruction(raw : uint16_t)
    + getRaw() : uint16_t
    + toHexString() : std::string
    + {abstract} execute(ctx : ExecutionContext&) : void
    # {static} nibble(raw : uint16_t, digit : int) : uint8_t
    # {static} lowByte(raw : uint16_t) : uint8_t
  }

  abstract class BinaryRegisterInstruction {
    # r : uint8_t
    # s : uint8_t
    # t : uint8_t
```

```
  + execute(ctx : ExecutionContext&) : void
  # {abstract} compute(a : uint8_t, b : uint8_t) : uint8_t
}

class LoadFromMemory {
  - r : uint8_t
  - address : uint8_t
  + execute(ctx : ExecutionContext&) : void
}
class LoadImmediate {
  - r : uint8_t
  - value : uint8_t
  + execute(ctx : ExecutionContext&) : void
}
class StoreToMemory {
  - r : uint8_t
  - address : uint8_t
  + execute(ctx : ExecutionContext&) : void
}
class CopyRegister {
  - r : uint8_t
  - s : uint8_t
  + execute(ctx : ExecutionContext&) : void
}
class AddTwosComplement {
  # compute(a : uint8_t, b : uint8_t) : uint8_t
}
class AddFloat {
  # compute(a : uint8_t, b : uint8_t) : uint8_t
}
class OrRegisters {
  # compute(a : uint8_t, b : uint8_t) : uint8_t
}
class AndRegisters {
  # compute(a : uint8_t, b : uint8_t) : uint8_t
}
class XorRegisters {
  # compute(a : uint8_t, b : uint8_t) : uint8_t
}
class RotateRight {
  - r : uint8_t
  - times : uint8_t
  + execute(ctx : ExecutionContext&) : void
}
class JumpIfEqual {
  - r : uint8_t
  - address : uint8_t
  + execute(ctx : ExecutionContext&) : void
}
class Halt {
  + execute(ctx : ExecutionContext&) : void
}

class InstructionDecoder <<factory>> {
  + {static} decode(high : uint8_t, low : uint8_t) : std::unique_ptr<Instruction>
```

```
    + {static} isValid(high : uint8_t, low : uint8_t) : bool
  }

  class "std::runtime_error" as RuntimeError <<std>>
  class InvalidInstructionException {
    - raw : uint16_t
    + InvalidInstructionException(raw : uint16_t)
    + raw() : uint16_t
  }
}

' ==================== LOADER ====================
package "Loader" {
  class ProgramLoader {
    + load(filename : const std::string&, startAddress : uint8_t, memory : Memory&) :
uint8_t
    - parseFile(filename : const std::string&) : std::vector<uint16_t>
    - parseLine(line : const std::string&) : std::optional<uint16_t>
    - checkStartAddress(startAddress : uint8_t, wordCount : size_t) : void
  }
  class ProgramLoadException {
    + ProgramLoadException(message : const std::string&)
  }
}

' ==================== FACADE ====================
class Machine <<facade>> {
  - screen : Screen
  - memory : Memory
  - cpu : CPU
  - loader : ProgramLoader
  - programStart : uint8_t
  + Machine()
  + loadProgram(filename : const std::string&, startAddress : uint8_t) : void
  + overwriteRegister(index : uint8_t, value : uint8_t) : void
  + overwriteMemory(address : uint8_t, value : uint8_t) : void
  + setPC(address : uint8_t) : void
  + setPCToProgramStart() : void
  + resetAll() : void
  + step() : bool
  + runAll() : void
  + isFinished() : bool
  + getMemory() : const Memory&
  + getCPU() : const CPU&
  + getScreen() : const Screen&
}

' ==================== UI ====================
package "UI" {
  class ConsoleUI {
    - machine : Machine&
    - renderer : ConsoleRenderer
    + ConsoleUI(machine : Machine&)
    + start() : void
    - showMenu() : void
```

```
    - handleLoadProgram() : void
    - handleOverwriteRegister() : void
    - handleOverwriteMemory() : void
    - handleSetPC() : void
    - handleRun() : bool
    - runAllAtOnce() : void
    - runStepByStep() : void
    - askExitOrLoadAnother() : bool
    - readHex(prompt : const std::string&, maxValue : unsigned = 0xFF) : uint8_t
    - readLine(prompt : const std::string&) : std::string
  }

  class ConsoleRenderer {
    + showState(machine : const Machine&) : void
    + showMemory(memory : const Memory&) : void
    + showRegisters(regs : const RegisterFile&) : void
    + showPCAndIR(pc : uint8_t, ir : uint16_t) : void
    + showScreen(screen : const Screen&) : void
    + showMessage(text : const std::string&) : void
    + showError(text : const std::string&) : void
  }
}

' ====================== RELATIONSHIPS ======================
' Ownership
Machine "1" *-- "1" Screen
Machine "1" *-- "1" Memory
Machine "1" *-- "1" CPU
Machine "1" *-- "1" ProgramLoader
CPU "1" *-- "1" RegisterFile
CPU "1" *-- "0..1" Instruction : current

' Associations
CPU "1" --> "1" Memory : reads / writes
Memory "1" --> "1" Screen : prints when\naddress = FF

' Interface realisation
CPU ..|> ExecutionContext

' Instruction hierarchy
Instruction <|-- LoadFromMemory
Instruction <|-- LoadImmediate
Instruction <|-- StoreToMemory
Instruction <|-- CopyRegister
Instruction <|-- BinaryRegisterInstruction
Instruction <|-- RotateRight
Instruction <|-- JumpIfEqual
Instruction <|-- Halt
BinaryRegisterInstruction <|-- AddTwosComplement
BinaryRegisterInstruction <|-- AddFloat
BinaryRegisterInstruction <|-- OrRegisters
BinaryRegisterInstruction <|-- AndRegisters
BinaryRegisterInstruction <|-- XorRegisters

' Dependencies
```

```
Instruction ..> ExecutionContext : uses
AddTwosComplement ..> ALU
AddFloat ..> ALU
OrRegisters ..> ALU
AndRegisters ..> ALU
XorRegisters ..> ALU
RotateRight ..> ALU
CPU ..> InstructionDecoder : decodes with
InstructionDecoder ..> Instruction : creates
InstructionDecoder ..> InvalidInstructionException : throws
ProgramLoader ..> InstructionDecoder : validates with
ProgramLoader ..> Memory : writes program to
ProgramLoader ..> ProgramLoadException : throws

' Exceptions
RuntimeError <|-- InvalidInstructionException
RuntimeError <|-- ProgramLoadException

' UI
ConsoleUI "1" --> "1" Machine
ConsoleUI "1" *-- "1" ConsoleRenderer
ConsoleRenderer ..> Machine : reads state of
@enduml
```